

# MoE Load Balancing & the All-to-All Bottleneck, Worked Out by Hand

WHY ROUTERS COLLAPSE, HOW THE AUXILIARY LOSS STOPS IT, AND WHY EXPERTS MUST SHARE A NODE

**What this document does.** It covers the two things that make MoE *work in production*, both by hand. First, the auxiliary load-balance loss: we route a small batch unevenly across four experts, compute  $L_{\text{balance}} = N \sum_i f_i P_i$ , and see it rise above its floor of 1 — the penalty that stops the router collapsing onto a favorite expert. Second, the expert-parallel all-to-all: we count the bytes moved per MoE layer and the NVLink-vs-InfiniBand latency gap that forces all experts onto one node. Every number comes from a short NumPy script (Appendix A, `m06_t05_moe_loadbalance_alltoall_engine.py`).

**The one idea.** Sparse routing has two failure modes. Statistically, the router wants to collapse onto a few experts (rich-get-richer); the balance loss fights that. Physically, the chosen experts live on *other GPUs*, so every MoE layer ships tokens across the network twice; that communication — not the math — becomes the bottleneck, and only NVLink is fast enough to hide it.

CONCEPTS COVERED, IN ORDER

mode collapse; the token-fraction  $f_i$  and mean-gate  $P_i$ ; the auxiliary loss  $L_{\text{balance}} = N \sum_i f_i P_i$  and why it pushes toward uniform; the all-to-all dispatch/combine pattern; the  $O(\text{batch} \cdot \text{seq} \cdot \text{hidden})$  byte cost; and the NVLink vs. inter-node latency wall.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

## Contents

|     |                                                             |   |
|-----|-------------------------------------------------------------|---|
| 1   | The problem — routers collapse                              | 2 |
| 2   | Step 1 — Measuring imbalance: $f_i$ and $P_i$               | 2 |
| 3   | Step 2 — The auxiliary load-balance loss                    | 3 |
| 4   | Step 3 — The other failure mode: experts live on other GPUs | 4 |
| 5   | Step 4 — The all-to-all byte cost                           | 4 |
| 6   | Step 5 — NVLink vs. inter-node: the latency wall            | 4 |
| 7   | The argument, at a glance                                   | 5 |
| 7.1 | Equation cheat-sheet                                        | 5 |
|     | Appendix A — Reference implementation                       | 5 |

## 1 The problem — routers collapse

Train an MoE on the task loss alone and the router learns a vicious cycle: the experts it happens to favor early get more tokens, more gradient, and improve faster — so they get favored even more, while the rest starve. Left unchecked the router routes everything to one or two experts and the other  $N - 2$  are dead weight. The model paid for  $N$  experts' capacity and uses two. This is **mode collapse**, and it defeats the entire point of MoE.

### Intuition

It is a popularity feedback loop. A new restaurant with one early five-star review gets more visitors, hence more reviews, hence more visitors — while an equally good restaurant with no early reviews stays empty forever. The router needs an external nudge that rewards *spreading the tokens around*, independent of which experts happen to look good first.

## 2 Step 1 — Measuring imbalance: $f_i$ and $P_i$

We route a batch of  $T = 8$  tokens across  $N = 4$  experts. Suppose expert 0 is “hot” and grabs five of the eight tokens; experts 1, 2, 3 get one each:

$$\text{assignment} = (0, 0, 0, 0, 0, 1, 2, 3).$$

The balance loss needs two per-expert statistics:

### Load and probability mass per expert

$$f_i = \frac{\#\{\text{tokens} \rightarrow \text{expert } i\}}{T} \text{ (load)}, \quad P_i = \frac{1}{T} \sum_t s_{t,i} \text{ (mean gate)}.$$

From the routing and the per-token router softmaxes (Appendix A):

$$f = (0.625, 0.125, 0.125, 0.125), \quad P = (0.507, 0.168, 0.170, 0.156).$$

$f_i$  counts *where tokens actually went* (a hard histogram);  $P_i$  averages *how much the router wanted each expert* (a soft signal). Expert 0 dominates both — 62.5% of tokens, 50.7% of the probability mass.

### Watch out

Why both  $f$  and  $P$ ?  $f_i$  is the quantity we truly want uniform, but it comes from a non-differentiable arg (top- $K$ ), so it carries no gradient.  $P_i$  is differentiable (it is built from the softmax). Multiplying them,  $f_i P_i$ , gives a term that is large exactly when an expert is both *over-used* and *over-wanted*, and has a usable gradient through  $P_i$ . That product is the clever core of the loss.

### 3 Step 2 — The auxiliary load-balance loss

#### Switch-Transformer auxiliary balance loss

$$L_{\text{balance}} = N \sum_{i=1}^N f_i P_i.$$

Added to the main loss with a small coefficient (0.01–0.05). Minimized when  $f_i = P_i = 1/N$  for all  $i$  (perfectly uniform).

#### Evaluate on the imbalanced batch

$$\sum_i f_i P_i = 0.625(0.507) + 0.125(0.168) + 0.125(0.170) + 0.125(0.156) = 0.3783,$$

$$L_{\text{balance}} = N \sum_i f_i P_i = 4 \times 0.3783 = 1.513. \quad \checkmark$$

Now the reference points that make 1.513 meaningful:

| Routing                                     | $f, P$                    | $L_{\text{balance}} = N \sum f_i P_i$                                    |
|---------------------------------------------|---------------------------|--------------------------------------------------------------------------|
| Perfect balance                             | $f_i = P_i = \frac{1}{4}$ | $4 \cdot 4 \cdot \frac{1}{4} \cdot \frac{1}{4} = \mathbf{1.000}$ (floor) |
| Our imbalanced batch                        | see above                 | 1.513                                                                    |
| Total collapse (all $\rightarrow$ expert 0) | $f_0=1, P_0 \approx .85$  | <b>3.400</b> (near worst)                                                |

#### Why the floor is 1 and uniform minimizes it

With  $\sum_i f_i = 1$  and  $\sum_i P_i = 1$ , the dot product  $\sum_i f_i P_i$  is minimized (subject to those simplex constraints) when both vectors are uniform,  $f_i = P_i = 1/N$ . Then

$$L_{\text{balance}} = N \cdot N \cdot \frac{1}{N} \cdot \frac{1}{N} = 1.$$

Any concentration of mass onto fewer experts makes the aligned  $f_i P_i$  terms larger than the spread-out ones, pushing the sum — and the loss — above 1. So gradient descent on  $L_{\text{balance}}$  literally pushes  $f$  and  $P$  toward the uniform  $1/N$ , i.e. toward using every expert equally.

#### How the loss prevents collapse

$L_{\text{balance}}$  is large precisely when load piles onto a few experts, and *shrinks toward its floor of 1 as routing approaches uniform*. Its gradient (through the differentiable  $P_i$ ) lowers the router's preference for the overloaded experts and raises it for the starved ones — directly counteracting the rich-get-richer loop. Our batch sits at 1.513, i.e. 0.513 of avoidable imbalance penalty above the floor; the optimizer will work that down.

#### Intuition

DeepSeek-V3's refinement: instead of a loss term (which slightly fights the task objective), add a per-expert *bias* to the router scores that is nudged up for underused experts and down for overused ones each step — with no gradient through the bias. Same balancing effect, no tug-of-war with the main loss. The accounting of  $f_i$  here is exactly what that bias controller watches.

## 4 Step 3 — The other failure mode: experts live on other GPUs

A 70B MoE has too many expert parameters to fit on one GPU, so we **shard experts across GPUs** (expert parallelism): GPU 0 holds experts 1,2; GPU 1 holds 3,4; and so on. But a token's hidden state arrives on whatever GPU is processing that *sequence*, while its chosen expert may live on a *different* GPU. So every MoE layer must:

1. **dispatch** — send each token's hidden state to the GPU owning its top- $K$  expert (an all-to-all);
2. run the experts locally (grouped GEMM);
3. **combine** — send each result back to the originating GPU (a second all-to-all).

Two all-to-all collectives per MoE layer, every layer, every token.

## 5 Step 4 — The all-to-all byte cost

Each token ships its hidden vector ( $d_{\text{model}}$  values, FP16) out and gets one back. So per dispatch the bytes moved scale as

### All-to-all payload per MoE layer

bytes  $\sim O(\text{batch} \times \text{seq} \times \text{hidden}) = B \cdot S \cdot d_{\text{model}} \cdot (\text{bytes/elt})$ ,  
moved *twice* (dispatch + combine) per MoE layer.

### Decode step, batch 32, $d_{\text{model}} = 5120$

Per-token hidden =  $5120 \times 2 \text{ B} = 10,240 \text{ B}$ . With  $B = 32$ ,  $S = 1$  (one new decode token per sequence):

dispatch payload =  $32 \times 1 \times 10,240 = 327,680 \text{ B} = 0.328 \text{ MB}$ ,

per MoE layer =  $2 \times 0.328 = 0.655 \text{ MB moved}$ .

With 60 MoE layers, a single forward pass performs  $2 \times 60 = \mathbf{120}$  all-to-all collectives. ✓

The byte volume per collective is modest — the killer is the *count* (120) multiplied by each collective's fixed *latency* floor.

## 6 Step 5 — NVLink vs. inter-node: the latency wall

An all-to-all's time at decode is set by its latency floor, not its bandwidth (the payloads are tiny). That floor depends entirely on the slowest link:

| Fabric                       | Per all-to-all                               | $\times 120$ per token |
|------------------------------|----------------------------------------------|------------------------|
| Intra-node <b>NVLink</b>     | $\sim 0.5\text{--}1.2 \text{ ms}$ (0.8 used) | <b>96 ms</b>           |
| Inter-node <b>InfiniBand</b> | $\sim 20\text{--}60 \text{ ms}$ (40 used)    | <b>4.8 s</b>           |

**The deployment rule**

120 all-to-alls per token is a fixed tax. On NVLink it costs  $\sim 96$  ms of communication — already significant but survivable. On inter-node InfiniBand the *same* 120 collectives cost  $\sim 4.8$  s *per token* — a  $\sim 50\times$  blow-up that makes the model unusable. Therefore: **the expert-parallel group must fit inside one NVLink domain** (8 GPUs on an H100/H200 node). Scale the dense layers with tensor parallelism if you must, but keep all the experts on one machine. DeepSeek-V3 runs exactly this on  $8\times$  H800 nodes.

**Watch out**

Even within a node, *hot experts* cause stragglers: if expert 0 receives  $2\text{--}3\times$  its share of tokens, its GPU runs longer and every other GPU idles waiting at the combine barrier. The balance loss of Steps 1–2 reduces this, but real traffic still skews; production fixes are a per-expert *capacity factor* (drop overflow tokens, residual still passes) and *replicating* the 2–3 hottest experts across GPUs to spread their load. These are latency-stability patches on top of the statistical balancing.

## 7 The argument, at a glance

| # | Step         | Result                                                          |
|---|--------------|-----------------------------------------------------------------|
| 1 | Measure      | $f = (.625, .125, .125, .125)$ , $P = (.507, .168, .170, .156)$ |
| 2 | Balance loss | $L = N \sum f_i P_i = 1.513$ (floor 1.0, collapse 3.4)          |
| 3 | Sharding     | experts on other GPUs $\Rightarrow$ 2 all-to-alls/layer         |
| 4 | Byte cost    | 0.328 MB/dispatch; 120 a2a/forward pass                         |
| 5 | Latency wall | NVLink 96 ms vs. inter-node 4.8 s ( $50\times$ )                |

### 7.1 Equation cheat-sheet

|                  |                                                              |
|------------------|--------------------------------------------------------------|
| Token fraction   | $f_i = \#\{\text{tokens} \rightarrow i\}/T$                  |
| Mean gate prob   | $P_i = \frac{1}{T} \sum_t s_{t,i}$                           |
| Balance loss     | $L_{\text{balance}} = N \sum_i f_i P_i \ (\geq 1)$           |
| All-to-all bytes | $\sim B \cdot S \cdot d_{\text{model}} \cdot (\text{B/elt})$ |
| Collectives/pass | $2 \times (\#\text{MoE layers})$                             |
| Comm/token       | $(\#\text{a2a}) \times (\text{per-a2a latency})$             |

## Appendix A — Reference implementation

Every number above is produced by `m06_t05_moe_loadbalance_alltoall_engine.py`. Run it to reproduce the balance-loss values and the all-to-all cost table; change the assignment, batch, or fabric latencies to explore.

```

1 """
2 Reference implementation for: Module 06 - Topic 5
3 "MoE load balancing (auxiliary loss) and the expert-parallel all-to-all cost."
4
5 Part A: an imbalanced routing of a small batch over N=4 experts; compute the
6 Switch-Transformer auxiliary balance loss  $L_{\text{balance}} = N \cdot \sum_i f_i \cdot P_i$ , and
7 contrast it with a perfectly balanced routing.
8
9 Part B: the all-to-all communication cost  $\sim O(\text{batch} \cdot \text{seq} \cdot \text{hidden})$  bytes per MoE

```

```

10 layer, and the NVLink (intra-node) vs InfiniBand (inter-node) latency gap that
11 forces the expert-parallel group to stay within one node.
12 """
13
14 import numpy as np
15 np.set_printoptions(precision=4, suppress=True)
16
17 # =====
18 # PART A - load balancing
19 # =====
20 N = 4                                # experts
21 # A batch of T=8 tokens, top-1 routing for clarity. Imbalanced assignment:
22 # expert 0 is a "hot" expert grabbing most tokens.
23 assign = np.array([0, 0, 0, 0, 0, 1, 2, 3]) # which expert each token went to
24 T = len(assign)
25 print("== PART A: load balancing ==")
26 print(f"N={N} experts, T={T} tokens, assignment =", assign)
27
28 # f_i = fraction of tokens routed to expert i.
29 f = np.array([(assign == i).mean() for i in range(N)])
30 print("\nf_i (fraction of tokens to expert i) =", f)
31
32 # P_i = mean router softmax probability on expert i across the batch.
33 # Construct per-token gate distributions that are consistent with the (skewed)
34 # routing: the chosen expert gets the largest prob.
35 gate_logits = np.array([
36     [2.5, 0.2, 0.1, 0.0], # -> 0
37     [2.2, 0.3, 0.0, 0.1], # -> 0
38     [2.8, 0.1, 0.2, 0.0], # -> 0
39     [2.0, 0.4, 0.1, 0.2], # -> 0
40     [2.4, 0.2, 0.3, 0.1], # -> 0
41     [0.3, 2.1, 0.2, 0.1], # -> 1
42     [0.2, 0.1, 2.3, 0.3], # -> 2
43     [0.1, 0.2, 0.3, 2.0], # -> 3
44 ])
45 def softmax_rows(Z):
46     Z = Z - Z.max(axis=1, keepdims=True)
47     E = np.exp(Z)
48     return E / E.sum(axis=1, keepdims=True)
49 probs = softmax_rows(gate_logits) # [T, N]
50 P = probs.mean(axis=0) # mean gate prob per expert
51 print("P_i (mean gate prob on expert i) =", P)
52
53 # Switch-Transformer auxiliary balance loss.
54 L_balance = N * np.sum(f * P)
55 print(f"\nL_balance = N * sum_i f_i * P_i = {N} * {np.sum(f * P):.5f} = {L_balance:.5f}")
56 print("(minimum possible value is 1.0 at perfect balance f_i=P_i=1/N)")
57
58 # Contrast: perfectly balanced routing (2 tokens each), uniform-ish probs.
59 f_bal = np.full(N, 1.0 / N)
60 P_bal = np.full(N, 1.0 / N)
61 L_bal_min = N * np.sum(f_bal * P_bal)
62 print(f"\nperfect balance: f_i=P_i=1/N -> L_balance = {L_bal_min:.5f}")
63 print(f"imbalance penalty above the floor = {L_balance - L_bal_min:.5f}")
64
65 # Show the collapse extreme: ALL tokens to expert 0.
66 f_collapse = np.array([1.0, 0, 0, 0])
67 P_collapse = np.array([0.85, 0.05, 0.05, 0.05])
68 L_collapse = N * np.sum(f_collapse * P_collapse)
69 print(f"total collapse (all to expert 0): L_balance = {L_collapse:.5f} (worst)")
70
71 # =====
72 # PART B - all-to-all communication cost

```

```

73 # =====
74 print("\n== PART B: expert-parallel all-to-all cost ==")
75 batch = 32
76 seq    = 1                # decode: 1 new token per sequence per step
77 hidden = 5120
78 bytes_fp16 = 2
79
80 # Each token's hidden state must be dispatched to its expert's GPU and the
81 # result combined back: payload ~ batch*seq*hidden*bytes, twice per MoE layer.
82 payload = batch * seq * hidden * bytes_fp16
83 print(f"per-token hidden = {hidden} x {bytes_fp16}B = {hidden*bytes_fp16} B")
84 print(f"dispatch payload (batch={batch}, seq={seq}) = {payload:,} B ")
85     f"= {payload/1e6:.3f} MB")
86 print(f"per MoE layer = 2 all-to-alls (dispatch + combine) = {2*payload/1e6:.3f} MB moved")
87
88 n_moe_layers = 60
89 print(f"per forward pass: {n_moe_layers} MoE layers x 2 = "
90       f"{2*n_moe_layers} all-to-alls")
91
92 # Latency: dominated by per-collective latency floor, not bandwidth, at decode.
93 nvlink_ms_per_a2a = 0.8      # intra-node NVLink, ~0.5-1.2 ms
94 ib_ms_per_a2a      = 40.0    # inter-node InfiniBand, ~20-60 ms
95 a2a_count = 2 * n_moe_layers
96 print(f"\nintra-node NVLink: {a2a_count} x {nvlink_ms_per_a2a} ms = "
97       f"{a2a_count*nvlink_ms_per_a2a:.1f} ms/token of comm")
98 print(f"inter-node IB      : {a2a_count} x {ib_ms_per_a2a} ms = "
99       f"{a2a_count*ib_ms_per_a2a/1000:.2f} s/token of comm <-- catastrophic")
100 print(f"slowdown factor inter/intra = {ib_ms_per_a2a/nvlink_ms_per_a2a:.0f}x")
101 print("\nRule: keep the expert-parallel group within ONE NVLink domain (<=8 GPUs).")

```

— END OF DOCUMENT —